Lesson 16

Struct

Struct-and-union notes
Textbook: section 3.0 Example 7

Slides – Vybihal & Bérubé-Vallières © 2023

Structures

- Structures (struct) are a collection of items that may have different types. Its elements are called members, and unlike arrays, they are accessed by name

Slides – Vybihal & Bérubé-Vallières © 2023

Example 1

- Here is how a structure might be declared as a global variable and its members accessed
  - In reality you would most likely use a structure tag, which we'll see later in this lecture

```
#include  <stdio.h>
#include <string.h>
struct  {
    char name[30];
    int age;
    float salary;
 } a;     // notice the semi-colon
 int main(void) {
    a.age = 25;
    strcpy(a.name, "John");
    printf("%s is %d years old\n", a.name, a.age);
}
```

members

Accessing members

Example 1

- Here is how a structure might be declared as a global variable and its members accessed
  - In reality you would most likely use a structure tag, which we'll see later in this lecture

```
#include  <stdio.h>
#include <string.h>
struct  {
      char name[30];
      int age;
      float salary;
 } a;     // notice the semi-colon
 int main(void) {
      a.age = 25;
      strcpy(a.name, "John");
      printf("%s is %d years old\n", a.name, a.age);
}
```

type

variable name

Same form of variable declaration as usual, type followed by variable name

Example 2

- Declaring multiple variables at the same time as global variables
  - In reality you would most likely use a structure tag, which we'll see later in this lecture

```
#include  <stdio.h>
#include <string.h>
struct  {
      char  name[30];
      int age;
      float salary;
 } a, b;
 int main(void) {
      a.age = 25;
      strcpy(a.name, "John");
      b.age = 30;
      strcpy(b.name, "Bob");
      printf("%s is %d years old, %s is %d years old\n", a.name, a.age, b.name, b.age);
} 
```

As usual, declarations can be of the form "type var_name1, var_name2, …"

Example 3

What if we needed to declare the same structure locally at different places, this would be cumbersome…:

```
#include <stdio.h>
#include <string.h>
int main(void) {
    struct  {
      char name[30];
      int age;
      float salary;
    } a;
    [...]
    struct  {
      char name[30];
      int age;
      float salary;
    } b;
}
```

There is a better way to do this, see next slide!

Example 4

Tags solve the problems from previous slide:

```
#include <stdio.h>
#include <string.h>
struct  PERSON {
        char name[30];
        int age;
        float salary;
 } ;   // notice the semi-colon even though no variable declaration
int main(void) {
      struct PERSON a;
        [...]
      struct PERSON b;
}
```

Tag

Tag replaces the members definitions

Slides – Vybihal & Bérubé-Vallières © 2023

Example 5

```
#include   <stdio.h>
#include  <string.h>
struct  PERSON {
        char name[30];
        int age;
        float salary;
 } d ;
int main(void) {
      struct PERSON a, b;
        [...]
      struct PERSON c;
}
```

Tag

Can still make global variable(s) at same time

Can still declare multiple variables at same time

Slides – Vybihal & Bérubé-Vallières © 2023

struct

Syntax of type struct:

```
struct OPTIONAL_TAG {
    MEMBERS;
};
```

Where:

- OPTIONAL_TAG- user-defined tag. If not present, we say the struct is anonymous
- MEMBERS
  - Is a list of semi-colon separated variable declarations
  - TYPE1 VAR1; TYPE2 VAR2; etc.
  - Note that we cannot initialize members from inside the struct definition (i.e. couldn't write int age = 0;)

Slides – Vybihal & Bérubé-Vallières © 2023

The dot operator

To access the fields within the structure we use the dot operator:

```
#include  <stdio.h>
#include <string.h>
struct PERSON {
    char name[30];
    int age;
    float salary;
 };
int main(void) {
      struct PERSON a;
      scanf("%s", a.name);
      scanf("%d", &(a.age));
      a.salary = 50.25;
      printf("%s %d %f", a.name, a.age, a.salary);
}
```

Example

```
#include <stdio.h>
struct PERSON { char name[30]; int age; float salary; } a, b, c; // 3 global variables
int main() {
    printf("Enter the name for person a: ");
    scanf("%s", a.name);
    printf("Enter the age: ");
    scanf("%d", &(a.age));
    printf("Enter the salary: ");
    scanf("%f", &(a.salary));
    // repeat the above for b and c
    printf("Enter the name for person b: ");
    scanf("%s", b.name);
}
```


Slides – Vybihal & Bérubé-Vallières © 2023

Struct assignment

Perhaps surprisingly, we can use the assignment operator with structs (unlike arrays) if they have compatible types (declared with same tag, or if anonymous, declared at same time)

```
#include <stdio.h>
struct COORDINATE {
    int x;
    int y;
};
int main(void) {
    struct COORDINATE a, b;
    a.x = 5;
    a.y = 7;
    b = a;
    printf("The coordinate of b is (%d, %d)\n", b.x, b.y);
}
```

Copies every member's value one by one (i.e. b.x = a.x; b.y = a.y;)

Struct assignment

Perhaps surprisingly, we can use the assignment operator with structs (unlike arrays) if they have compatible types (declared with same tag, or if anonymous, declared at same time)

```
#include <stdio.h>
struct COORDINATE {
    int x;
    int y;
};
int main(void) {
    struct COORDINATE a, b;
    a.x = 5;
    a.y = 7;
    b = a;
    printf("The coordinate of b is (%d, %d)\n", b.x, b.y);
}
```

Copies every member's value one by one (i.e. b.x = a.x; b.y = a.y;)

Perhaps even more surprisingly, this would work with arrays in side of structs! So this would work:
struct {int arr[5];} a, b;
a.arr[0] = 5;
b=a;
printf("%d\n", b.arr[0]); \\ outputs 5

Nested structures

Structures can be used as members

```
#include <stdio.h>
#include <string.h>
struct  NAME {
        char first[30];
        char last [30];
 };
struct  PERSON {
        struct NAME name;
        int age;
        float salary;
};
void print_name(struct NAME n) {
      printf("%s %s\n", n.first, n.last);
}
int main(void) {
      struct PERSON a;
      strcpy(a.name.first,   "John");
      strcpy(a.name.last,  "Smith");
      print_name(a.name); //
}
```

See how structs can be used as arguments

Chained dot operators

Typedef and structs

Often, people will use typedef in combination with structs. This way, the "struct" keyword doesn't have to be used

```
#include <stdio.h>

#include <string.h>

typedef struct {
        char name[30];
        int age;
        float salary;
 } Person;
int main(void) {
        Person a, b;
        [...]
        Person c;
}
```

Remember that typedef is used like:
typedef type new_name

Since this is part of a typedef, "Person" is not a variable declaration, it's the new name for the type

Notice how the struct keyword is not used anymore

Array of struct

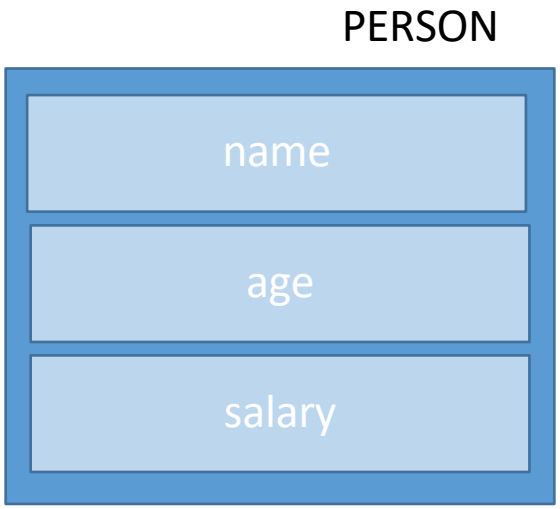
```
#include <stdio.h>
struct PERSON { char name[30]; int age; float salary; } ;
int main() {
    struct PERSON people[100];
    for(int x=0; x<100; x++) {
        scanf("%s", people[x].name);
        scanf("%d", &(people[x].age));
        scanf("%f", &(people[x].salary));

        if (people[x].age == 20) printf("Hey you are 20!!\n");
    }
}
```

Sizeof and structs

In memory, members of a structs are placed in sequence

```
struct PERSON {
    char name[30];  // char *name;
    int age;
    float salary;
};
```



However, computers often require addresses of certain items to be on certain multiples (For example, could say that integers are always on multiples of 4). Because of this, the compiler can keep unused bytes between elements. As such, sizeof might be bigger than expected, since it will contain these unused bytes

Recap

- Syntax of type struct:

  struct OPTIONAL_TAG { MEMBERS;

  } OPTIONAL-Variables;

  Typedef struct { MEMBERS;

  } NEW-TYPE-NAME;

  - You can pass structs by value or by reference as arguments to functions, and you can also return them.

Recap

- Using structs makes it much more convenient to move arrays around as a single unit.

- The size of a struct is not simply the sum of the sizes of its fields as padding may be added for memory alignment.

Practice problem

Write a C program that does the following:

- Define a struct Coord type representing a coordinate in a 2d space (X and Y).
- Implement a function read_coords(int *capacity) that allocates a dynamic array named 'CoordArray' of such struct Coord type of a capacity of user choice. The function should also read coordinate inputs from the user and store them in the dynamically allocated CoordArray.
- Implement a function Coord centroid(Coord *arr, int capacity) that computes the centroid (average X and Y) of all coordinates entered.
- In the main function, output the computed centroid to the user.